OS

Definition

An operating system (OS) is the software layer that acts as an interface between user applications and the computer hardware. Its primary functions include:

- **Resource Management:** Allocates CPU time, memory, and storage to various processes.
- **Process Management:** Creates, schedules, and terminates processes to ensure efficient multitasking.
- **Memory Management:** Controls access to RAM, ensuring that each process operates within its own isolated address space.
- **File System Management:** Organizes and manages data stored on disks, providing a structure for file access.
- **I/O Device Control:** Provides a unified interface for applications to interact with hardware devices like keyboards, monitors, and network adapters.
- **Security & Protection:** Regulates user access through authentication and guards against unauthorized system interaction.

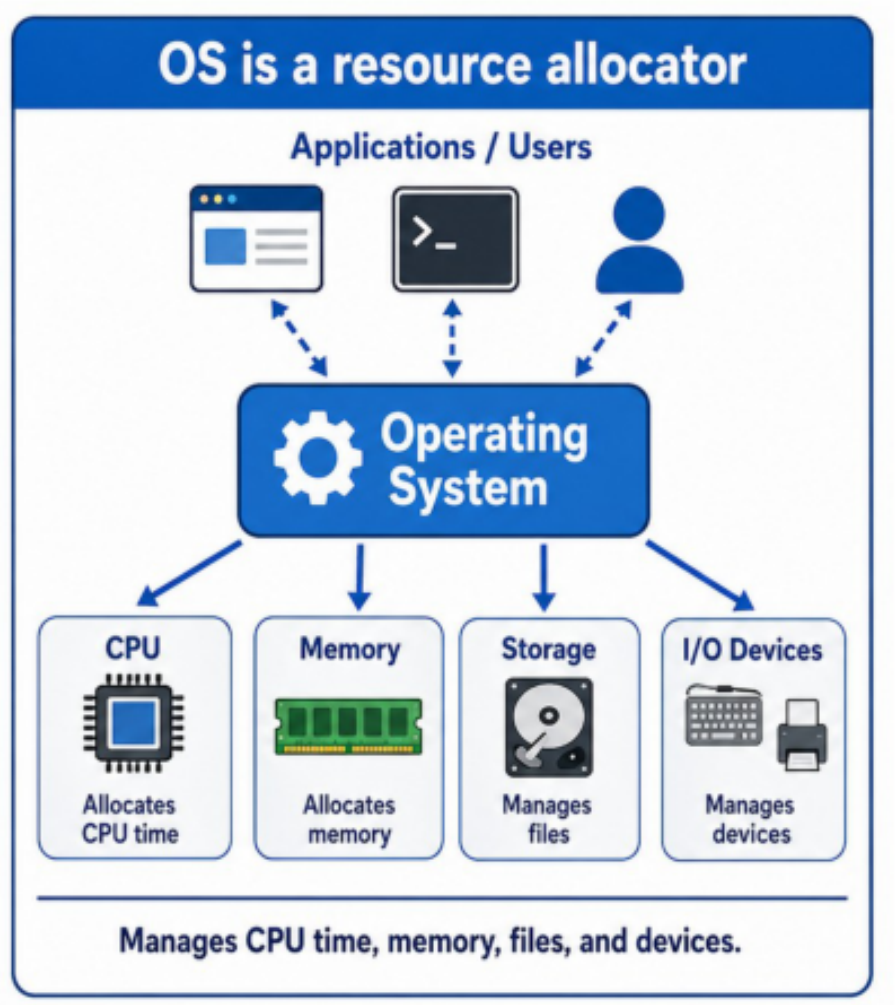

Figure 1: OS is a resource allocator

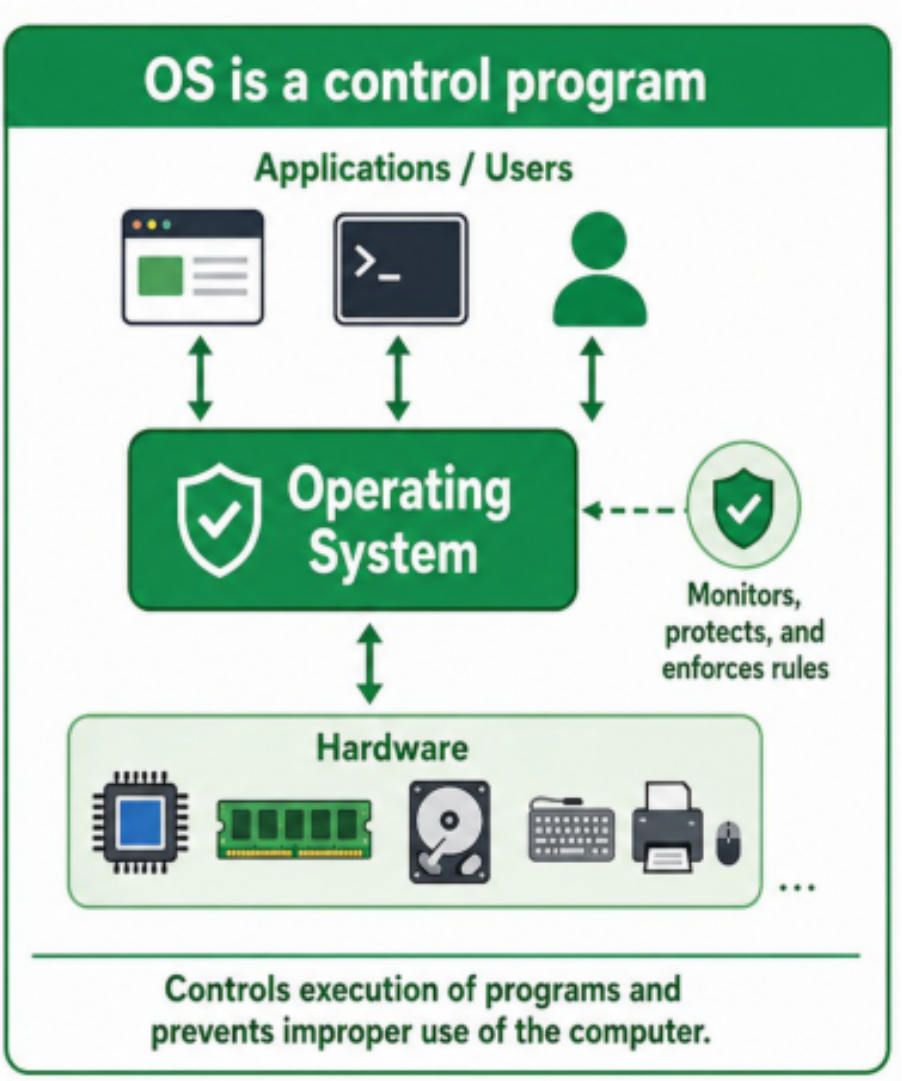

Figure 2: OS is a control program

OSTEP defines it in three steps which are:

- Virtualization
- Concurrency
- Parallelism

Computer System Structure

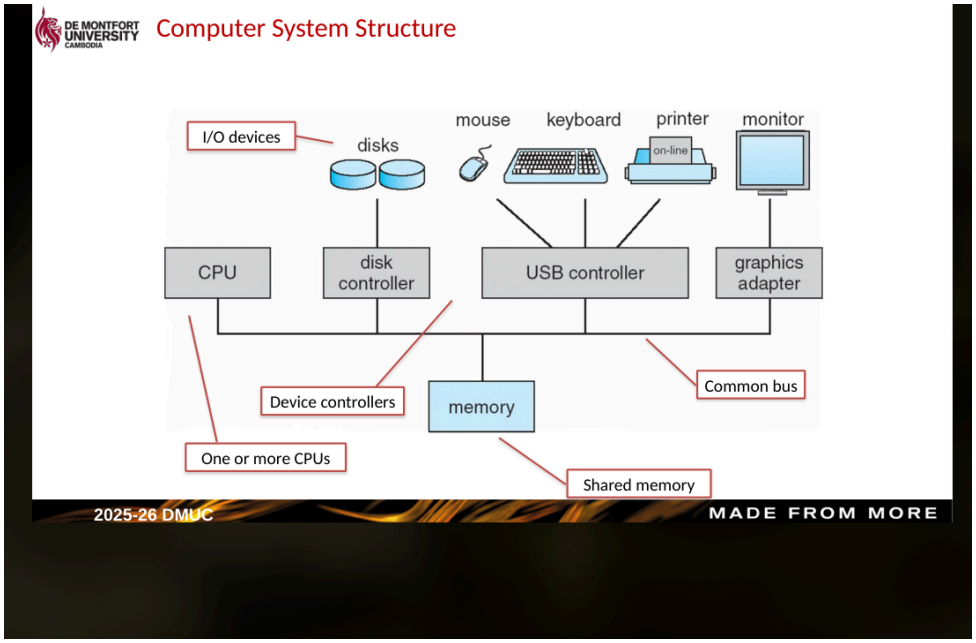

Figure 3: Computer System Structure

Interrupts

- OS is interrupt driven - an interrupt is a signal sent to the OS:
  - Hardware interrupt by one of the devices
  - Software interrupt generated by user request
  - Trap or exception generated by an error

Dual-Mode Operation

- Dual-mode operation allows the OS to protect itself, as well as other components.
  - User mode and kernel mode

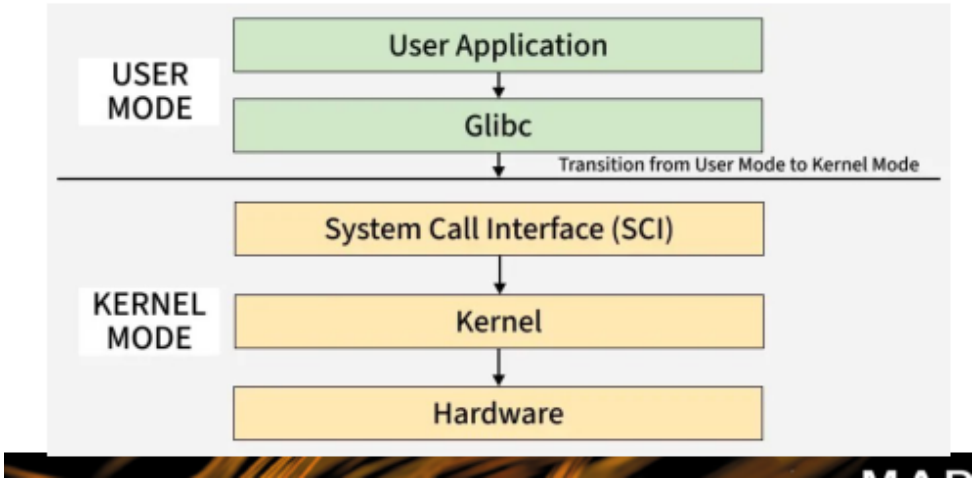

Figure 4: User mode and Kernel mode

Kernel Mode

- In kernel mode, the operating system has unrestricted access to the underlying hardware and memory.
  - Can execute privileged instructions that are not available to user programs.

- Can manage system resources such as CPU scheduling, memory allocation, and I/O devices.
- Can handle interrupts, exceptions, and traps directly.
- Executes with full authority to modify the system state, including kernel data structures.

User Mode

- In user mode, the operating system restricts the program's access to hardware and system resources to prevent system crashes or malicious activity.
  - Programs have limited access to memory and cannot directly access I/O devices.
  - Privileged instructions are prohibited; any attempt to execute them triggers a trap or exception that the kernel must handle.
  - Programs must request system services via system calls to perform restricted operations.
  - The mode provides a protected environment where applications can run safely without interfering with the kernel or other running processes.

Comparison

Feature	User Mode	Kernel Mode
Access	Restricted	Unrestricted
Instructions	Non-privileged only	Privileged and non-privileged
Memory	Isolated address space	Full system memory
Services	Request via system calls	Direct access to hardware
Stability	Safe/Protected	Critical/Requires care

Kernel Architectures

- **Monolithic Kernel**: All OS services (file system, device drivers, memory management, etc.) run in the same kernel address space. This provides high performance due to minimal communication overhead, but a bug in one component can crash the entire system.
  - Examples: Linux, FreeBSD, MS-DOS.
- **Microkernel**: Only the most essential functions (minimal mechanisms for IPC, virtual memory, and scheduling) run in the kernel mode. Other services run in user space as separate processes. This design is highly modular and stable, though communication between services can induce latency.
  - Examples: QNX, L4, MINIX.
- **Hybrid Kernel**: A compromise between monolithic and microkernel designs. It maintains the core performance of a monolithic system while incorporating some modularity from a microkernel, often by running certain services (like graphics drivers) in user space while keeping others in the kernel.
  - Examples: Windows NT (and its successors like Windows 10/11), macOS (XNU kernel).
- We are doing Linux afterwords, woohoo!

Open Source OSes

- An open source OS makes its source code available for inspection, modification, and redistribution under an open source licence.
- Examples include:

1. Linux distributions such as Ubuntu, Debian, Fedora, and Arch.
2. FreeBSD, OpenBSD, and NetBSD.
3. MINIX.

- Open source matters because users and organizations can audit code, adapt it to their needs, and contribute fixes upstream.

System Calls

- A system call is the controlled entry point from a user program into the kernel.
- User programs cannot directly perform privileged operations such as raw disk access, process creation, or direct device control.
- Instead, they request kernel services through syscalls.
- Examples:
  1. `read()` and `write()` for file or device I/O.
  2. `open()` and `close()` for files.
  3. `fork()`, `exec()`, `wait()`, and `exit()` for process control.
  4. `chmod()` and `chown()` for permission and ownership changes.
- Syscalls are normally wrapped by libraries, so programmers often call library functions rather than invoking the trap instruction directly.

Linux Permissions

- File owners should be able to control what can be done to a file and who can do it.
- Linux records access rights for three classes:
  1. **Owner**: the user who owns the file.
  2. **Group**: users in the file's group.
  3. **World/other**: everyone else.
- The three basic permissions are:
  1. `r` read: view file contents or list directory names.
  2. `w` write: modify a file or create/delete entries in a directory.
  3. `x` execute: run a file or enter/search a directory.
- Example from `ls -l`:

```
-rw-r--r-- 1 ali staff 4.9K 6 Oct 11:44 archive-file.zip
drwxr-xr-x 46 ali staff 1.5K 8 Oct 14:28 architecture
-rwxr-xr-x 1 ali staff 71K 26 Oct 18:57 schedule.ods
```

- The first character shows file type:
  1. `-` normal file.
  2. `d` directory.
- The next nine characters are permissions in groups of three:
  1. owner, group, other.
- Numeric permission values:
  1. read = `4`
  2. write = `2`
  3. execute = `1`
- Add the values for each class:
  1. `7` = `rwx`
  2. `6` = `rw-`
  3. `5` = `r-x`
  4. `4` = `r--`
  5. `0` = `---`
- Example: `chmod 761 game` means:

1. owner = 7 = `rwx`
2. group = 6 = `rw-`
3. other = 1 = `--x`

- Symbolic forms can be clearer:

```
chmod u+rwx game
chmod g+rw game
chmod o+x game
chgrp groupname filename
usermod -a -G groupname username
```

Common Linux Commands

Command	Use
`cd`	Change directory.
`mkdir`	Create a directory.
`rmdir`	Remove an empty directory.
`cat`	Print or concatenate file contents.
`vi`	Open the `vi` text editor.
`tree`	Display a directory tree.
`top`	Show running processes and resource usage.
`grep`	Search text for matching lines.
`chmod`	Change file or directory permissions.
`xclock`	Open a simple X11 clock application, useful for testing GUI forwarding.

Bash

- Bash is a common Unix/Linux shell, usually located at `/bin/bash`.
- A shell reads commands, starts programs, expands variables, redirects input and output, and runs scripts.
- Common Bash features:
  1. pipes, such as `cat file.txt | grep error`.
  2. redirection, such as `command > output.txt`.
  3. environment variables, such as `$PATH`.
  4. scripts starting with a shebang like `#!/bin/bash`.

Processes

- A **program** is a passive file stored on disk.
- A **process** is a program in execution.
- A program becomes a process when the executable is loaded into memory.
- One program can have many processes, such as several users running the same command at the same time.
- A process needs resources:
  1. CPU time.
  2. memory.

3. files.
4. I/O devices.
5. initialization data.

- The OS is responsible for creating, deleting, suspending, resuming, and scheduling processes.

Parts of a Process in Memory

- **Text section**: program code/instructions.
- **Data section**: global and static variables.
- **Heap**: dynamically allocated memory during runtime.
- **Stack**: function parameters, return addresses, and local variables.
- **Program counter**: address of the next instruction to execute.
- **Registers**: CPU state used by the running process.

The stack usually grows downward and the heap usually grows upward. The unused space between them can remain unmapped until needed.

Process States

- `new`: process is being created.
- `ready`: process is waiting to be assigned to a CPU.
- `running`: instructions are currently executing.
- `waiting`: process is blocked waiting for an event, I/O, or resource.
- `terminated`: process has finished.

Typical transitions:

- `new -> ready`: process is admitted by the OS.
- `ready -> running`: scheduler dispatches the process.
- `running -> ready`: CPU is preempted or time slice expires.
- `running -> waiting`: process requests I/O or waits for an event.
- `waiting -> ready`: event completes.
- `running -> terminated`: process exits.

Process Control Block

- The PCB, also called a task control block, stores the OS's record of a process.
- It includes:
  1. process state.
  2. process ID.
  3. program counter.
  4. CPU registers.
  5. scheduling information and priority.
  6. memory-management information.
  7. accounting information such as CPU time used.
  8. I/O status and open files.
- During a context switch, the OS saves the old process state into its PCB and loads the next process state from its PCB.
- Context switching is overhead because the CPU is not doing useful application work during the switch.
- In Linux, process information can be inspected under `/proc/<pid>/`, such as:
  1. `/proc/<pid>/status`
  2. `/proc/<pid>/sched`
  3. `/proc/<pid>/maps`
  4. `/proc/<pid>/stack`

Process Creation and Termination

- Processes form a tree:
  1. A parent process creates child processes.
  2. Children can create more children.
  3. Each process is managed using a PID.
- Common Unix/Linux system calls:
  1. `fork()` creates a child process that is initially a copy of the parent.
  2. `exec()` replaces the process memory image with a new program.
  3. `wait()` lets a parent wait for a child to finish and collect its status.
  4. `exit()` terminates the current process.
- A **zombie** process has terminated, but its parent has not yet collected its exit status with `wait()`.
- An **orphan** process continues running after its parent has terminated.

Virtual Memory

- Processes use **logical** or **virtual** addresses generated by the CPU.
- RAM uses **physical** addresses.
- The memory-management unit (MMU) maps virtual addresses to physical addresses at runtime.
- This separation is important because:
  1. each process can have its own private address space.
  2. a process does not need to know where it is stored in physical RAM.
  3. the OS can protect processes from accessing each other's memory.
  4. programs can be larger than physical memory.

Swapping

- Swapping temporarily moves a process or parts of a process out of RAM to a backing store on disk, then brings it back when needed.
- This allows the total memory used by processes to exceed physical RAM.
- Swap time is mostly transfer time, so swapping large processes can make context switching extremely slow.
- Example: swapping out 100 MB at 50 MB/s takes about 2 seconds; swapping in another 100 MB takes another 2 seconds.

Benefits of Virtual Memory

- Only the needed parts of a program have to be in memory.
- More programs can run concurrently.
- Less I/O is needed when loading or swapping programs.
- Address spaces can contain holes for stack/heap growth.
- Shared libraries can be mapped into multiple processes.
- Shared memory can be implemented by mapping the same physical pages into more than one process.
- `fork()` can be faster because pages can initially be shared and copied only when modified.